

```
In [43]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
In [44]: df = pd.read_csv("gym_members_exercise_tracking.csv")
df
```

Out[44]:

	Age	Gender	Weight (kg)	Height (m)	Max_BPM	Avg_BPM	Resting_BPM	Session_Duration (hours)	Calories_Burned	Workout_Type	Fat_Percentage	Water_Intake (liters)
0	56	Male	88.3	1.71	180	157	60	1.69	1313.0	Yoga	12.6	3.5
1	46	Female	74.9	1.53	179	151	66	1.30	883.0	HIIT	33.9	2.1
2	32	Female	68.1	1.66	167	122	54	1.11	677.0	Cardio	33.4	2.3
3	25	Male	53.2	1.70	190	164	56	0.59	532.0	Strength	28.8	2.1
4	38	Male	46.1	1.79	188	158	68	0.64	556.0	Strength	29.2	2.8
...	...	...	...	...	...	...	...	...	...	...	...	...
968	24	Male	87.1	1.74	187	158	67	1.57	1364.0	Strength	10.0	3.5
969	25	Male	66.6	1.61	184	166	56	1.38	1260.0	Strength	25.0	3.0
970	59	Female	60.4	1.76	194	120	53	1.72	929.0	Cardio	18.8	2.7
971	32	Male	126.4	1.83	198	146	62	1.10	883.0	HIIT	28.2	2.1
972	46	Male	88.7	1.63	166	146	66	0.75	542.0	Strength	28.8	3.5

973 rows × 15 columns



```
In [45]: #Vérification des valeurs manquantes
df.isnull().sum()
```

```
Out[45]: Age                0
Gender                0
Weight (kg)          0
Height (m)           0
Max_BPM              0
Avg_BPM              0
Resting_BPM          0
Session_Duration (hours) 0
Calories_Burned      0
Workout_Type         0
Fat_Percentage       0
Water_Intake (liters) 0
Workout_Frequency (days/week) 0
Experience_Level     0
BMI                 0
dtype: int64
```

```
In [46]: #Vérification des valeurs aberrantes
col = ['Age', 'Weight (kg)', 'Height (m)', 'Max_BPM', 'Avg_BPM', 'Resting_BPM', 'Session_Duration (hours)', 'Calories_Burned', 'Fat_Percentage', 'Water_Intake (liters)']
df_clean = df.copy()
for col in col:
    Q1 = df_clean[col].quantile(0.25)
    Q3 = df_clean[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    df_clean = df_clean[(df_clean[col] >= lower_bound) & (df_clean[col] <= upper_bound)]
print("Taille avant :", len(df))
print("Taille après :", len(df_clean))
df=df_clean
```

Taille avant : 973
Taille après : 931

```
In [47]: X = df.drop(columns=['Calories_Burned'])
y = df['Calories_Burned']
```

```
In [48]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
```

```
In [49]: cat_cols = X.select_dtypes(include=['object', 'category']).columns
num_cols = X.select_dtypes(include=['int64', 'float64']).columns
```

```
In [50]: #Encodage
le = LabelEncoder()
for col in cat_cols:
    X_train[col] = le.fit_transform(X_train[col])
    X_test[col] = le.transform(X_test[col])
```

```
In [51]: #Standardisation
scaler = StandardScaler()
```

```
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])
```

decesion tree

```
In [ ]: class Node :
    def __init__(self, feature=None, threshold=None, left=None, right=None, \
        impurity=None, n_samples=None, is_leaf=False, value=None):
        self.feature=feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.is_leaf=is_leaf
        self.value = value
        self.impurity = impurity
        self.n_samples = n_samples

class DecesionTreeRegression:
    def __init__(self, min_samples_split=2, min_samples_leaf=1, \
        min_impurity_decrease=0.0, max_depth = None):
        self.max_depth=max_depth
        self.min_samples_split = min_samples_split
        self.min_samples_leaf = min_samples_leaf
        self.min_impurity_decrease=min_impurity_decrease
        self.root = None

    def calculatemse(self, y):
        if len(y) == 0:
            return 0
        return np.mean((y-np.mean(y))**2)

    def calcule_impurity(self, y):
        #on remplace ici mse par gini si ona une classification
        return self.calculatemse(y)

    def best_split(self, X, y):
        n_samples=len(y)
        best_feature, best_threshold, best_gain=None, None, float('-inf')
        current_impurty = self.calcule_impurity(y)
        for feature in range(X.shape[1]):
            thresholds = np.unique(X[:, feature])
            for threshold in thresholds:
                left_mask = X[:, feature] <= threshold
                right_mask = ~left_mask
                if (len(y[left_mask]) < self.min_samples_leaf) or (len(y[right_mask]) < self.min_samples_leaf):
                    continue
                left_impurity = self.calcule_impurity(y[left_mask])
                right_impurity = self.calcule_impurity(y[right_mask])
                weighted_impurity = (len(y[left_mask])*left_impurity + len(y[right_mask])*right_impurity)/n_samples
                impurity_gain = current_impurty-weighted_impurity
                if (impurity_gain>best_gain):
                    best_feature=feature
                    best_threshold=threshold
                    best_gain=impurity_gain
        return best_feature, best_threshold

    def fit(self, X, y):
        X = np.array(X)
        y = np.array(y)
        self.root = self.build_tree(X, y)
        return self

    def build_tree(self, self, X, y, depth=0):
        n_samples=len(y)
        node_impurity = self.calcule_impurity(y)
        node_value = np.mean(y)
        node = Node(value=node_value, impurity=node_impurity, is_leaf=True, n_samples=n_samples)
        if (
            (self.max_depth is None or depth < self.max_depth)
            and len(y) >= self.min_samples_split
            and len(np.unique(y)) > 1
        ):
            best_feature, best_threshold = self.best_split(X, y)
            if (best_feature is not None):
                left_mask = X[:, best_feature] <= best_threshold
                right_mask = ~left_mask
                X_left, y_left, X_right, y_right = X[left_mask], y[left_mask], X[right_mask], y[right_mask]
                left_node = self.build_tree(X_left, y_left, depth + 1)
                right_node = self.build_tree(X_right, y_right, depth + 1)

                node.feature = best_feature
                node.threshold = best_threshold
                node.left = left_node
                node.right = right_node
                node.is_leaf = False
        return node

    def _predict_one(self, self, x):
        node = self.root
        while node is not None and not node.is_leaf:
            if x[node.feature] <= node.threshold:
                node = node.left
            else:
                node = node.right
        return node.value if node is not None else None

    def predict(self, self, X):
        X = np.array(X)
```

```
return np.array([self._predict_one(sample) for sample in X])
```

```
In [ ]: class RandomForestRegressor:
    # Une forêt aléatoire = ensemble d'arbres de décision avec agrégation par vote majoritaire

    def __init__(self, n_estimators=10, max_depth=10, max_features=None):
        # n_estimators : nombre d'arbres à entraîner (par défaut 10)
        self.n_estimators = n_estimators
        # max_depth : profondeur maximale de chaque arbre
        self.max_depth = max_depth
        # max_features : nombre de features à utiliser pour chaque arbre
        # (None = toutes les features)
        self.max_features = max_features
        # trees : liste pour stocker les arbres entraînés
        self.trees = []

    def bootstrap_sample(self, X, y):
        # Bootstrap = rééchantillonnage avec remplacement
        # Crée un sous-ensemble aléatoire de la même taille que l'ensemble original
        # Cela permet à chaque arbre d'avoir des données légèrement différentes
        n_samples = X.shape[0]
        # Tire aléatoirement n_samples indices (avec remplacement)
        indices = np.random.choice(n_samples, size=n_samples, replace=True)

        return X[indices], y[indices]

    def fit(self, X, y):
        # Entraîne tous les arbres de la forêt
        self.trees = []
        n_features = X.shape[1]
        # Utilise max_features si défini, sinon utilise toutes les features
        max_features = self.max_features if self.max_features else n_features

        # Entraîne n_estimators arbres
        for i in range(self.n_estimators):
            # Crée un nouvel arbre
            tree = DecisionTreeRegression(max_depth=self.max_depth, min_samples_split=2, min_samples_leaf=1)

            # Crée un échantillon bootstrap des données
            X_sample, y_sample = self.bootstrap_sample(X, y)

            # Sélectionne aléatoirement max_features features pour cet arbre
            selected_features = np.random.choice(n_features, size=max_features, replace=True)

            # Extrait seulement les features sélectionnées
            X_sample_subset = X_sample[:, selected_features]

            # Entraîne l'arbre sur les données bootstrap avec les features sélectionnées
            tree.fit(X_sample_subset, y_sample)

            # Stocke l'arbre et les features utilisées (utile pour la prédiction)
            self.trees.append((tree, selected_features))

    def predict(self, X):
        tree_predictions = np.array([tree.predict(X[:, features]) for tree, features in self.trees])
        mean_predictions = np.mean(tree_predictions, axis=0)
        return mean_predictions
```

```
In [ ]: X_train = X_train.values
X_test = X_test.values
y_train = y_train.values
y_test = y_test.values
```

```
In [59]: from sklearn.metrics import mean_squared_error, r2_score

model = RandomForestRegressor(n_estimators=40, max_depth=7)
model.fit(X_train, y_train)
prediction = model.predict(X_test)
mse = mean_squared_error(y_test, prediction)
print("R2:\n ", r2_score(y_test, prediction))
```

```
R2:
0.9685252540526241
```

```
In [64]: from sklearn.ensemble import RandomForestRegressor as SklearnRandomForest
# Importe la métrique de précision
from sklearn.metrics import accuracy_score, r2_score
# Crée une instance de Random Forest de sklearn avec paramètres par défaut
clf = SklearnRandomForest()
# Entraîne le modèle sklearn
# .ravel() transforme y_train en vecteur 1D pour éviter un warning
clf.fit(X_train, y_train.ravel())
# Fait les prédictions avec le modèle sklearn
y_sklearn_pred = clf.predict(X_test)
# Calcule la précision du modèle sklearn
print("R2:\n ", r2_score(y_test, y_sklearn_pred))
```

R2:
0.9757276121509045

```
In [60]: # ===== SAUVEGARDE DU MODÈLE ENTRAÎNÉ =====  
# Importe le module pickle pour sérialiser/désérialiser les objets Python  
import pickle  
  
# Ouvre un fichier en mode écriture binaire ('wb')  
# 'with' ferme automatiquement le fichier à la fin  
with open('random_forest_regression.pkl', 'wb') as f:  
    # Sérialise le modèle et le sauvegarde dans le fichier  
    pickle.dump(model, f)  
  
# Affiche un message de confirmation  
print("Modèle Random Forest sauvegardé dans 'random_forest_model.pkl'")
```

Modèle Random Forest sauvegardé dans 'random_forest_model.pkl'

```
In [61]: # ===== CHARGEMENT DU MODÈLE SAUVEGARDÉ =====  
# Ouvre le fichier du modèle en mode lecture binaire ('rb')  
with open('random_forest_regression.pkl', 'rb') as f:  
    # Désérialise l'objet modèle depuis le fichier  
    loaded_model = pickle.load(f)  
  
# Maintenant 'loaded_model' est identique au 'model' d'avant la sauvegarde  
# Vous pouvez l'utiliser pour faire de nouvelles prédictions sans réentraîner
```